

Antarctic Weather Platform

Technical Report

A backend and frontend system for retrieving, caching, aggregating,
and visualizing historical Antarctic meteorological observations
from AEMET OpenData

Mária Nyolcas

Prepared for GS Inima Environment

Development period: 21–28 August 2026

Abstract

This report documents the engineering reasoning behind a backend and frontend system for retrieving, caching, aggregating, and visualizing historical Antarctic meteorological observations from AEMET OpenData, built for GS Inima Environment's evaluation of a hypothetical wind-farm project in Antarctica. It complements the repository's `README.md`, which covers setup, usage, and a summary of design decisions, by explaining *why* each non-trivial decision was made, including the alternatives considered and their trade-offs. Source excerpts are referenced by file path and line range against the repository at the time of writing, so every claim here can be checked directly against running code, not reconstructed from memory.

A note on scope and timing

This report is being written incrementally, alongside development, rather than after the fact: a deliberate choice made explicit here because it carries a real trade-off. Sections describing finished, tested subsystems (the AEMET integration, timezone and DST handling, aggregation, and SQLite caching) are written against stable code. Sections depending on work not yet complete at the time of a given revision (frontend architecture, CI, final trade-offs) are marked with a visible `TODO` rather than filled with speculative or placeholder content, and are completed once the corresponding code exists. This is stated plainly so the report is never mistaken for describing functionality that has not been verified.

Contents

1	Executive Summary	3
2	Requirements Analysis	3
2.1	Source of truth	3
2.2	Core functional requirements, as specified	3
2.3	Ambiguities identified before implementation	4
2.4	Assumptions requiring verification	4
3	Architecture	4
3.1	Layering, and the requirement it serves	4
3.2	Orchestration and dependency injection	5
3.3	Three model shapes, not one	6

1 Executive Summary

The system exposes a single primary endpoint, `GET /observations`, that retrieves historical weather observations for one of two Spanish Antarctic research stations (Gabriel de Castilla, Juan Carlos I) over a caller-specified date range, optionally aggregated to hourly, daily, or monthly resolution, and returns temperature, pressure, and wind speed in a consistent, timezone-normalized shape.

Three engineering concerns dominated the design, in the order the challenge itself weights them:

1. **Correctness under real-world data conditions.** AEMET’s actual API behavior, including a two-step response indirection, a "NaN" string sentinel for missing measurements, an undocumented data-quality flag, and station-dependent optional fields, was verified against the live API before any client code was written, rather than assumed from the endpoint’s description alone (Section ??).
2. **Temporal correctness.** The system distinguishes an instant, a local wall-clock representation, a named timezone’s DST rules, and a fixed UTC offset as four genuinely different concepts, and handles the two DST edge cases (nonexistent and ambiguous local times) explicitly rather than relying on Python’s implicit defaults (Section ??).
3. **A caching strategy whose behavior is fully explainable.** Rather than a general interval-coverage algorithm, the cache uses a deliberately scoped single-range containment check, reasoned through as a trade-off between completeness and explainability (Section ??).

2 Requirements Analysis

2.1 Source of truth

The authoritative requirements are the official challenge specification (PDF, three parts) together with direct clarification from the hiring contact. Where the two could be read differently, the original PDF is treated as authoritative, since it is the actual assessment document rather than a paraphrase of it.

2.2 Core functional requirements, as specified

- A Python web API (FastAPI or Flask) exposing the AEMET Antarctic endpoint’s data.
- Caller-specified start and end datetime (AAAA-MM-DDTHH:MM:SS).
- An optional location/timezone parameter, expressible either as an IANA timezone name (e.g. `Europe/Berlin`) or as a fixed UTC offset (e.g. `+02:00`).
- A choice of exactly two supported stations: “Gabriel de Castilla” and “Juan Carlos I”.
- A time-aggregation selector: None, Hourly, Daily, or Monthly, with daily/monthly aggregation required to respect the meteo station’s timezone rather than grouping raw UTC timestamps.
- Zero to three requested measurement types (temperature, pressure, wind speed); zero selected means all three are returned.
- Output datetimes in `Europe/Madrid` (CET/CEST), always carrying an explicit UTC offset.
- Explicit confirmation of DST behavior.
- SQLite persistence to avoid redundant upstream requests, with attention to database best practice, motivated explicitly by an anticipated high-volume, latency-sensitive consumer (intraday traders).
- Structured logging for troubleshooting.
- Automated tests, not required to be exhaustive.

- A frontend allowing the data to be queried and viewed in a table, optionally a chart, with explicit permission to use any reasonable technology stack.

2.3 Ambiguities identified before implementation

The specification is precise about behavior but silent on several implementation-level questions that materially affect correctness. Each was resolved deliberately, documented as an assumption, and isolated behind a narrow interface so it can be revisited without a wider rewrite.

Aggregation statistic. The specification requires aggregation but does not name a statistic. Arithmetic mean was chosen for all three measurements (Section ??); a second question — whether wind speed specifically should also report a maximum, given the wind-farm context — was raised with the assigning team rather than decided unilaterally, since it changes the response contract.

Default input timezone. The specification marks the timezone parameter optional without stating a default. **Europe/Madrid** was chosen over UTC specifically because it keeps an unspecified date range aligned with the same calendar day in the (unconditionally Madrid) output, rather than spilling across a day boundary — a concrete failure mode that was worked through with a numeric example before the decision was made.

Data-quality flag. AEMET’s live response includes a `qdato` field (data quality, 0 = good, 1 = bad) that is not part of the specification’s field table. Its existence was discovered only by making a live API call, not by reading documentation. Whether a flagged-bad reading should be silently excluded from aggregation or exposed unfiltered was likewise raised with the assigning team.

Cache staleness. The specification states AEMET’s Antarctic dataset updates only a few times a year, but does not say whether a previously cached range should ever be considered stale and re-fetched. The current implementation caches a fetched range indefinitely; whether a revalidation window is expected was raised with the assigning team rather than assumed.

2.4 Assumptions requiring verification

Two further points were resolved by direct experimentation against the live AEMET API rather than left as assumptions, and are recorded here because they materially shaped the client and domain model:

- Whether the AEMET Antarctic endpoint returns data directly or via an indirection (a documented risk called out by the assigning team) — confirmed to be a two-step indirection (Section ??).
- The exact station identifiers for the two required stations, and whether the field schema is uniform across them — confirmed via AEMET’s own OpenAPI specification and a live request per station; the two stations were found *not* to return an identical field set (Section ??).

3 Architecture

3.1 Layering, and the requirement it serves

The backend is organized into five layers, each with one responsibility:

Layer	Responsibility
API / Routes	Request validation, HTTP concerns, response shaping
Application / Service	Orchestration: cache check → AEMET fetch → aggregate
Domain	Pure functions: timezone conversion, aggregation, quality filtering
Repositories / Integrations	AEMET client, SQLite repository
External systems	SQLite, AEMET OpenData

This is not layering for its own sake. The concrete requirement it serves is testability under the specific correctness risks this system has: the domain layer (`app/domain/time.py`, `app/domain/aggregation.py`) has *no* dependency on FastAPI, SQLAlchemy, or `httpx`, so the timezone and DST logic — the single riskiest part of this system, per Section ?? — can be unit tested by constructing plain Python `datetime` values and asserting on plain return values, without a database, an HTTP mock, or a running application. The same is true of the aggregation module. A change to how AEMET responses are parsed, or how the API formats a response, cannot silently change how a DST transition is handled, because the code paths do not intersect.

3.2 Orchestration and dependency injection

The application/service layer is a single class, `WeatherService` (`backend/app/services/weather_service.py`), whose entire responsibility is sequencing: check whether the requested range is cached, call AEMET only on a miss, persist the result, then aggregate and return. It depends on its collaborators through its constructor rather than constructing them itself:

```

1 class AemetObservationSource(Protocol):
2     """The subset of AemetClient this service needs, lets tests substitute a
3         fake with no real HTTP client behind it."""
4
5     async def get_observations(
6         self, station: Station, start: datetime, end: datetime
7     ) -> list[StationObservation]: ...
8
9 class WeatherService:
10     """Orchestrates cache lookup, AEMET retrieval, and aggregation."""
11
12     # Exceptions from either collaborator propagate unchanged; mapping
13     # them to HTTP responses is the API layer's job, not this one's.
14     def __init__(
15         self, aemet_client: AemetObservationSource, repository:
16             ObservationRepository
17     ) -> None:
18         self._aemet_client = aemet_client
19         self._repository = repository

```

Listing 1: `WeatherService` depends on an AEMET source through a `Protocol`, not the concrete client class. `backend/app/services/weather_service.py`, lines 12–29.

This is a direct application of dependency inversion, chosen deliberately over the alternative of having `WeatherService` import and instantiate `AemetClient` directly. The concrete alternative that was rejected — constructing the real client inside the service and mocking `httpx` at the transport level in every service test — would work, but it means every service-layer test also exercises (and can be broken by) HTTP mocking concerns that have nothing to do with the orchestration logic actually being tested. Depending on a `Protocol` means the test suite’s fake client (`FakeAemetClient`, `tests/unit/test_weather_service.py`, lines 16–27) is a small class with no knowledge of HTTP at all, and — because `Protocol` is structurally typed — `mypy` verifies that both the fake and the real `AemetClient` satisfy the same interface, so this is a property enforced by the type checker, not merely true by convention.

3.3 Three model shapes, not one

A single AEMET observation is represented by three distinct types as it moves through the system, and this distinction is deliberate rather than incidental:

1. **Transport DTO** (`AemetObservationDTO`, `app/integrations/aemet/schemas.py`) — mirrors AEMET’s actual wire format, including its quirks (Section ??).
2. **Domain model** (`StationObservation`, same file) — a frozen, verified, in-domain reading with no knowledge of AEMET’s field names or serialization conventions.
3. **API response model** (`ObservationResponse`, `app/api/schemas.py`) — the wire contract this application exposes, independent of both AEMET’s format and the internal domain representation.

Collapsing these into one type would couple three independently-changing concerns: AEMET could alter its response format, the internal representation could be refactored for a new aggregation strategy, and the public API contract could evolve for frontend needs — none of which should require touching the other two.